



Trento Apartment Service (TAS)

Deliverable D2

Gruppo #4

Mikele Golemi	243170
Leonardo Tartini	245119
Francesco Zadra	244544

26 aprile 2026

Indice

1	Diagramma UML dei componenti	2
1.1	Diagramma dei componenti	2
1.2	Descrizione dei componenti	2
2	Diagramma UML delle classi	8
2.1	Classi di supporto	8
2.2	Gestione degli utenti	10
2.3	Pubblicizzazione appartamento	11
2.4	Gestione interna appartamento	13
2.5	Diagramma completo	14
3	Constraint OCL	17
3.1	Contratto - Validità delle date	17
3.2	Appartamento - Coerenza studentsOnly	17
3.3	Recensione - Range valutazione	17
3.4	Amministratore - Univocità pIVA	17
3.5	Guasto - Priorità e stato	17
3.6	Bolletta - Periodo valido	18
3.7	Camera - Prezzo positivo	18
3.8	Login - Utente esistente	18
3.9	Creazione Contratto - Disponibilità appartamento	18
3.10	Registrazione - Creazione utente	18

1 Diagramma UML dei componenti

In questa sezione viene presentato il diagramma UML dei componenti di una prima progettazione architeturale del progetto Trento Apartment Service. Vengono successivamente descritti nello specifico tutti i vari componenti, evidenziando le interfacce e il loro scopo.

1.1 Diagramma dei componenti

In questa sottosezione viene illustrato il diagramma UML dei componenti.

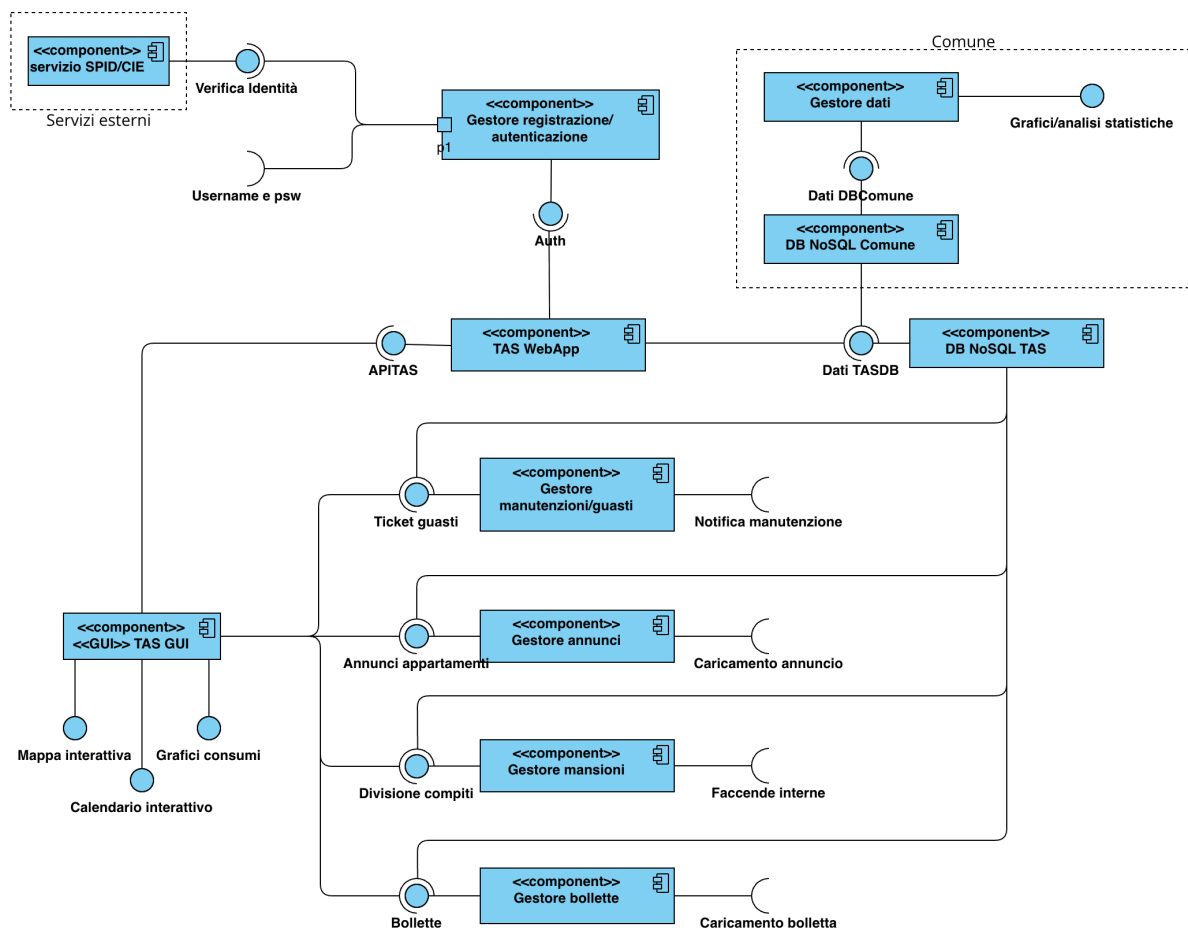


Figura 1: Diagramma dei componenti – Trento Apartment Service

1.2 Descrizione dei componenti

In questa sottosezione si descrivono i componenti, mettendone in evidenza lo scopo e le interfacce.

Servizio SPID/CIE

Questo componente gestisce l'autenticazione sicura e affidabile tramite SPID o CIE.

Servizio SPID/CIE		
Tipologia	Nome	Descrizione
Fornita	Verifica identità	Il componente fornisce un'interfaccia per la verifica dell'identità (RF8, RF35)

Gestore registrazione/autenticazione

Questo componente gestisce le richieste di autenticazione e di registrazione al portale TAS tramite servizio esterno di autenticazione (SPID/CIE) e tramite username e password.

Gestore registrazione/autenticazione		
Tipologia	Nome	Descrizione
Richiesta	Verifica identità	Il componente utilizza un'interfaccia fornita dal servizio esterno SPID/CIE per la verifica dell'identità (RF8, RF35)
Richiesta	Username e psw	Il componente utilizza un'interfaccia che permette di accedere al sistema TAS tramite uso di username e password
Fornita	Auth	Il componente fornisce un'interfaccia di autenticazione per l'accesso alla WebApp

TAS WebApp

TAS WebApp è il componente centrale del sistema che orchestra le richieste degli utenti autenticati, instradandole verso i gestori specializzati (annunci, manutenzioni, bollette, consumi) e coordinando l'accesso al database.

TAS WebApp		
Tipologia	Nome	Descrizione
Richiesta	Auth	Il componente richiede un'interfaccia di autenticazione per l'accesso alla WebApp
Richiesta	Dati TASDB	Il componente richiede un'interfaccia che permetta di accedere ai dati del database del sistema
Fornita	APITAS	APITAS è l'interfaccia REST che TAS WebApp espone verso TAS GUI per l'accesso alle funzionalità del sistema

TAS GUI

Questo componente fornisce all'utente l'interfaccia grafica e visiva del sistema.

TAS GUI		
Tipologia	Nome	Descrizione
Richiesta	APITAS	Il componente richiede l'interfaccia REST che TAS WebApp espone per l'accesso alle funzionalità del sistema
Richiesta	Ticket guasti	Il componente richiede un'interfaccia per le operazioni di aggiornamento e visualizzazione dei ticket di segnalazione (RF18, RF19, RF25)
Richiesta	Annunci appartamenti	Il componente richiede un'interfaccia per la visualizzazione degli annunci immobiliari (RF0, RF6, RF22, RF23)
Richiesta	Divisione compiti	Il componente richiede un'interfaccia che esponga la gestione delle faccende domestiche condivise tra coinquilini
Richiesta	Grafici consumi	Il componente richiede un'interfaccia che esponga la visualizzazione grafica dei dati energetici
Fornita	Mappa interattiva	Il componente fornisce un'interfaccia che espone la visualizzazione geografica degli annunci immobiliari (RF0)
Fornita	Calendario interattivo	Il componente fornisce un'interfaccia che espone la gestione del calendario condiviso per le faccende domestiche (RF15)
Fornita	Grafici consumi	Il componente fornisce un'interfaccia che espone graficamente i dati delle bollette riferite a una specifica locazione (RF17)

Gestore manutenzioni/guasti

Questo componente gestisce il ciclo di vita dei ticket di segnalazione guasti (apertura, aggiornamento dello stato, archiviazione).

Gestore manutenzioni/guasti		
Tipologia	Nome	Descrizione
Richiesta	Notifica manutenzione	Il componente richiede un'interfaccia di notifica di guasti e segnalazione di problemi durante la locazione (RF18)
Fornita	Ticket guasti	Il componente fornisce un'interfaccia che espone le operazioni di apertura e aggiornamento dei ticket di segnalazione

Gestore annunci

Questo componente gestisce la pubblicazione, la modifica e la ricerca degli annunci immobiliari.

Gestore annunci		
Tipologia	Nome	Descrizione
Richiesta	Caricamento annuncio	Il componente richiede un'interfaccia che permetta agli amministratori di caricare e pubblicare annunci immobiliari (RF20, RF23)
Fornita	Annunci appartamenti	Il componente fornisce un'interfaccia che espone la ricerca e la visualizzazione degli annunci immobiliari (RF10)

Gestore mansioni

Questo componente gestisce la suddivisione delle faccende domestiche tra inquilini, inclusa la gestione del calendario condiviso e delle liste della spesa.

Gestore mansioni		
Tipologia	Nome	Descrizione
Richiesta	Faccende interne	Il componente richiede un'interfaccia che permetta agli inquilini di una stessa locazione di caricare le faccende interne (RF14)
Fornita	Divisione compiti	Il componente fornisce un'interfaccia che espone la divisione delle faccende domestiche condivise tra i coinquilini (RF15)

Gestore bollette

Questo componente è dedicato alla gestione delle bollette legate ad una locazione.

Gestore bollette		
Tipologia	Nome	Descrizione
Richiesta	Caricamento bolletta	Il componente richiede un'interfaccia che permetta all'amministratore di caricare le bollette (acqua, luce e gas) relative a una locazione
Fornita	Bollette	Il componente fornisce un'interfaccia che espone le bollette legate alla locazione caricate dall'amministratore (RF24)

DB NoSQL TAS

Questo componente è un database persistente del sistema TAS che archivia i dati applicativi, ticket, annunci, bollette, consumi, contratti e mansioni.

DB NoSQL TAS		
Tipologia	Nome	Descrizione
Richiesta	Ticket guasti	Il componente richiede un'interfaccia per le operazioni di aggiornamento e visualizzazione dei ticket di segnalazione (RF18, RF19, RF25)
Richiesta	Annunci appartamenti	Il componente richiede un'interfaccia per la visualizzazione degli annunci immobiliari (RF0, RF6, RF22, RF23)
Richiesta	Divisione compiti	Il componente richiede un'interfaccia che esponga la gestione delle faccende domestiche condivise tra coinquilini
Richiesta	Bollette	Il componente richiede un'interfaccia che esponga le bollette legate alla locazione caricate dall'amministratore (RF24)
Fornita	Dati TASDB	Il componente fornisce un'interfaccia che permette di accedere ai dati del database del sistema

DB NoSQL Comune

Questo componente è un database dedicato al comune che contiene dati aggregati e anonimizzati relativi al mercato immobiliare.

DB NoSQL Comune		
Tipologia	Nome	Descrizione
Richiesta	Dati TASDB	Il componente richiede un'interfaccia che permetta di accedere ai dati del database del sistema TAS
Fornita	Dati DBComune	Il componente fornisce un'interfaccia che espone l'accesso ai dati aggregati e anonimizzati del mercato immobiliare (RF29)

Gestore dati

Questo componente appartiene al sottosistema del Comune e si occupa di aggregare i dati del mercato immobiliare per statistiche e analisi del territorio.

Gestore dati		
Tipologia	Nome	Descrizione
Richiesta	Dati DBComune	Il componente richiede un'interfaccia che esponga l'accesso ai dati aggregati e anonimizzati del mercato immobiliare (RF29)
Fornita	Grafici/analisi statistiche	Il componente fornisce un'interfaccia che espone report, grafici e statistiche del mercato immobiliare di Trento (RF30, RF33)

2 Diagramma UML delle classi

In questa sezione viene presentato il diagramma UML delle classi con le relazioni che le interconnettono. Sono specificati attributi e operazioni principali per ogni classe.

N.B. In fase di implementazione verranno eseguite ottimizzazioni ed aggiunte necessarie per la gestione concreta del DB¹, come ad esempio l'inserimento di ID univoci nelle classi dove manca un attributo identificativo, in modo da ridurre il passaggio di interi oggetti come parametri delle funzioni.

2.1 Classi di supporto

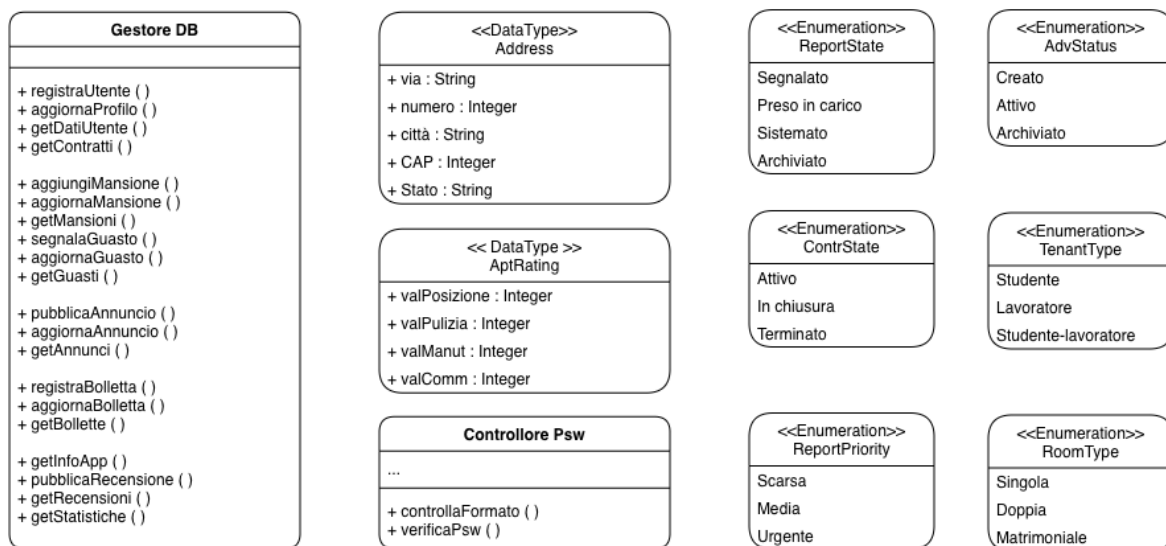


Figura 2: Classi che definiscono DataType specifici, enumerazioni e funzionalità globali

Controllore Psw

Questa classe si occupa di verificare la correttezza formale delle password durante registrazione e login.

Operazione	Descrizione
controllaFormato (String psw)	Verifica che il formato della password inserita in fase di registrazione rispetti i vincoli minimi di complessità per garantire sicurezza.
verificaPsw (String psw)	Effettua l'hashing della password inserita durante il login e verifica la corrispondenza con quella registrata per l'email fornita.

Gestore DB

La classe del Gestore DB implementa tutte le funzioni che richiedono un accesso al DB, le quali vengono invocate dalle diverse classi in base a permessi e necessità.

¹* DB = DataBase

Operazione	Descrizione
registraUtente (Utente u)	Salva nel DB i dati inseriti dall'utente in fase di registrazione.
aggiornaProfilo (Utente u)	Sostituisce dati/foto profilo vecchi dell'utente con quelli nuovi inseriti nella pagina di gestione profilo.
getDatiUtente (String email)	Una volta fatto il login, carica sulla piattaforma i dati dell'utente
getContratti (String email)	Carica i contratti associati all'utente loggato.
aggiungiMansione (Mansione m)	Registra nel DB la mansione aggiunta da un inquilino, con nome e data inseriti, più eventuali oggetti (e.g. per lista della spesa).
aggiornaMansione (Mansione m)	Contrassegna la mansione come svolta.
getMansioni (String email)	Carica le mansioni associate all'inquilino.
segnalaGuasto (Guasto g)	Registra nel DB il guasto segnalato da un inquilino.
aggiornaGuasto (Guasto g)	Aggiorna lo stato del guasto una volta preso in carico o risolto, attribuendo anche i valori anche alle rispettive date.
getGuasti (Appartamento a)	Carica lo storico di guasti e manutenzioni dell'appartamento gestito da/in cui risiede l'utente loggato.
pubblicaAnnuncio (Annuncio a)	Registra nel DB l'annuncio pubblicato da un amministratore.
aggiornaAnnuncio (Annuncio a)	Aggiorna lo stato o modifica un annuncio esistente secondo i nuovi dati inseriti dall'amministratore nella pagina di modifica annuncio.
getAnnunci (...)	Carica tutti gli annunci attivi e li rende visibili all'utente. La stessa funzione può ricevere in input diversi parametri (prezzo, zona, tipo stanza ...) per filtrare la ricerca.
registraBolletta (Bolletta b)	Registra nel DB la bolletta caricata dall'amministratore.
aggiornaBolletta (Bolletta b)	Contrassegna la bolletta come pagata una volta saldato l'importo.
getBollette (Appartamento a)	Carica lo storico delle bollette e consumi dell'appartamento gestito da/in cui risiede l'utente loggato.
getInfoApp (Annuncio a)	Dall'annuncio selezionato, la funzione carica tutti i dati di un appartamento per mostrarli all'utente che effettua la ricerca.
pubblicaRecensione (Recensione r)	Registra nel DB la recensione pubblicata da un inquilino.
getRecensioni (Appartamento a)	Carica tutte le recensioni pubblicate dagli inquilini che riguardano l'appartamento selezionato.
getStatistiche (...)	Carica i dati aggregati e anonimizzati degli appartamenti rientranti nei parametri di ricerca, per mostrare statistiche e metriche ai dipendenti comunali.

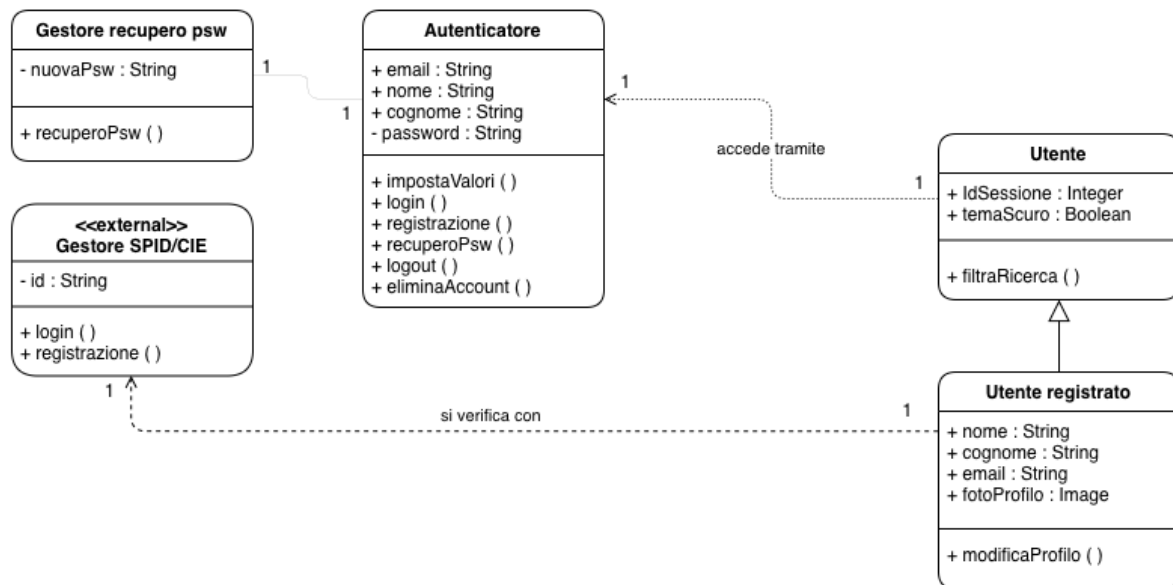


Figura 3: Classi legate alla registrazione degli utenti

2.2 Gestione degli utenti

Utente

La classe Utente fa riferimento agli utilizzatori anonimi della piattaforma, che hanno solo la possibilità di visualizzare e ricercare annunci.

Operazione	Descrizione
filtraRicerca (...)	L'utente può applicare diversi filtri alla ricerca, come la zona, il numero e il tipo di stanze, la fascia di prezzo...

Autenticatore

La classe Autenticatore si occupa dell'intero ciclo di vita di un Utente, dalla registrazione all'eventuale cancellazione dell'account.

Operazione	Descrizione
impostaValori (...)	Legge i valori inseriti nei campi di registrazione/login e li salva nelle variabili corrispondenti.
login (String email, String psw)	Verifica l'esistenza di un account associato alla mail inserita. Se trova una corrispondenza controlla la correttezza della password e accede, altrimenti chiama <i>registrazione()</i>
registrazione (Utente u)	Chiama l'apposita funzione del Gestore DB per registrare un nuovo utente su Trento Apartment Service.
recuperoPsw (String email)	Avvia il procedimento di recupero password
logout ()	Chiude la sessione attiva sul dispositivo e disconnette l'account precedentemente collegato.
eliminaAccount ()	Effettua il <i>logout ()</i> ed elimina l'account associato alla mail in uso fino ad allora.

Gestore recupero psw

Questa classe si occupa del procedimento di recupero dell'accesso all'account associato alla propria mail.

Operazione	Descrizione
recuperoPsw (String email)	Invia un link alla mail inserita e porta l'utente ad una pagina dove poter inserire una nuova password.

Utente registrato

Una volta effettuata la registrazione tramite l'autenticatore, l'utilizzatore diventa un Utente registrato, con associati i dati inseriti.

Operazione	Descrizione
modificaProfilo (Utente u)	Registra le modifiche apportate ai dati personali di un utente e le invia al Gestore DB per salvarle.

Gestore SPID/CIE

La classe Gestore SPID/CIE si occupa della verifica dell'identità di un utente registrato, condizione necessaria per poter essere registrato come inquilino di un appartamento.

Operazione	Descrizione
registrazione ()	Associa l'identità digitale certificata (verificata tramite piattaforma esterna) all'account attualmente attivo sul dispositivo, per un periodo di tempo limitato.
login ()	Permette all'utente di riconfermare la propria identità per prolungare la sessione .

2.3 Pubblicizzazione appartamento

Amministratore

L'amministratore registratosi alla piattaforma ha il controllo su uno o più appartamenti, che può promuovere tramite la pubblicazione di annunci.

Operazione	Descrizione
pubblicaAnnuncio (Annuncio a)	Inserisce i dati necessari e chiama la funzione del Gestore DB per pubblicare l'annuncio.

Annuncio

Questa classe fa riferimento agli annunci pubblicati dagli amministratori per promuovere i propri appartamenti.

Operazione	Descrizione
modificaAnnuncio (Annuncio a)	Aggiorna stato o descrizione dell'annuncio secondo quanto indicato dall'amministratore nell'apposita pagina.

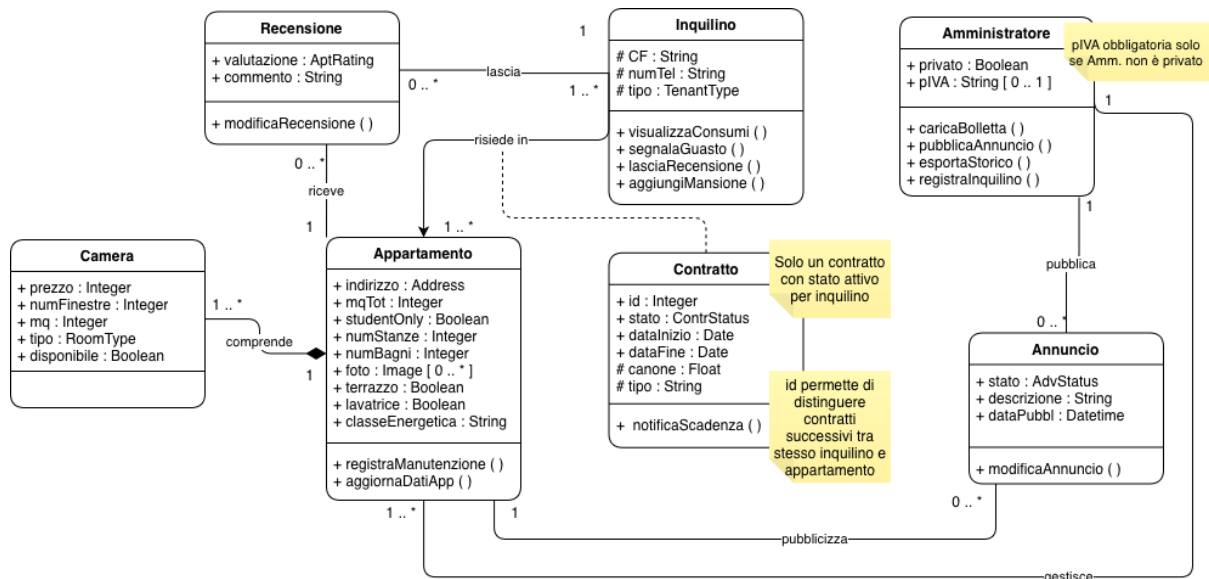


Figura 4: Classi che riguardano la pubblicizzazione di un appartamento e i contratti che lo legano agli inquilini.

Inquilino

La classe Inquilino riguarda gli utenti registrati, verificati tramite SPID/CIE e che hanno un contratto di locazione attivo registrato a loro nome.

Operazione	Descrizione
lasciaRecensione (Recensione r)	Registra la valutazione dell'appartamento più un eventuale commento e chiama la funzione del Gestore DB per pubblicare la recensione.

Recensione

Questa classe formalizza le recensioni pubblicate dagli inquilini sugli immobili in cui hanno risieduto. Essa prevede una valutazione di diversi aspetti ed un eventuale commento.

Operazione	Descrizione
modificaRecensione (Recensione r)	Aggiorna le valutazioni o il commento di una recensione precedentemente pubblicata da un inquilino.

Appartamento

Questa classe contiene tutti i dati riguardanti un appartamento, che comprende una o più camere.

Operazione	Descrizione
registraManutenz (Guasto g)	Comunica al Gestore DB la risoluzione di un guasto precedentemente segnalato e aggiorna lo storico manutenzioni.
aggiornaDatiApp (Appartamento a)	Sostituisce i dati riguardanti l'appartamento con quelli nuovi inseriti dall'amministratore nella pagina di gestione appartamento.

Contratto

Questa classe contiene i dati principali dei contratti che legano inquilini ed appartamenti. Essa non è in relazione con la classe *Amministratore* per i seguenti motivi: 1) TAS non si occupa della stesura dei contratti e non analizza le questioni legali che relazionano inquilini, proprietari ed eventuali agenzie; 2) non sempre il locatore/proprietario registrato nel contratto coincide con l'amministratore effettivo dell'appartamento.

Operazione	Descrizione
notificaScadenza ()	Il sistema notifica l'inquilino per metterlo al corrente riguardo l'avvicinamento della data di fine contratto.

2.4 Gestione interna appartamento

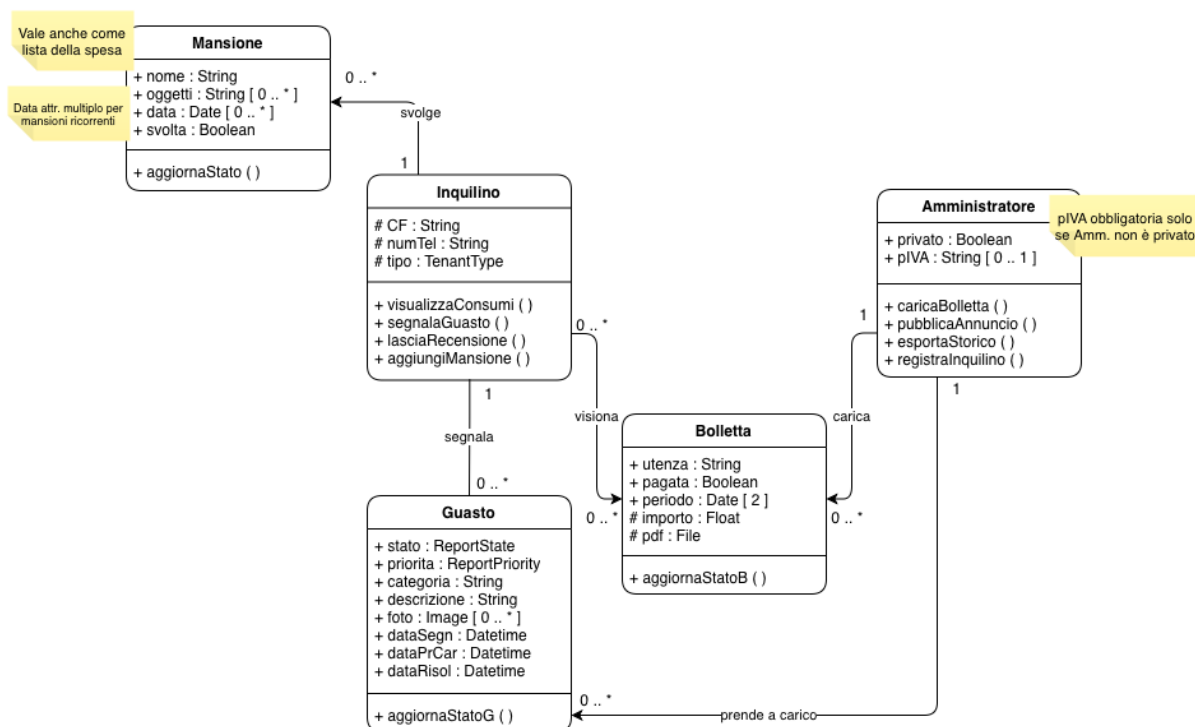


Figura 5: Classi legate alle funzionalità specifiche degli inquilini e la loro interazione con l'amministratore.

Amministratore

L'amministratore, oltre a pubblicare annunci, si occupa di tenere sotto controllo lo stato dei suoi appartamenti.

Operazione	Descrizione
caricaBolletta (Bolletta b)	Inserisce i dati richiesti e chiama la funzione del Gestore DB per registrare la bolletta.
esportaStorico (Appartamento a)	Genera un report periodico con costi medi e manutenzioni riguardanti un appartamento sotto la gestione dell'amministratore.
registraInquilino (Contratto c)	Registra un utente (verificato con SPID/CIE) come inquilino di un dato appartamento secondo quanto stabilito nel contratto.

Inquilino

La gestione interna della casa è principalmente nelle mani dei coinquilini, i quali possono monitorare i consumi, assegnare e dividersi le faccende o segnalare guasti all'amministratore.

Operazione	Descrizione
visualizzaConsumi (Appartamento a)	Richiede al DB la visualizzazione delle bollette legate all'appartamento in cui risiede, più grafici dell'andamento medio per periodo.
segnalaGuasto (Guasto g)	Inserisce i dati necessari e chiama la funzione del Gestore DB per salvare la segnalazione di un guasto.
aggiungiMansione (Mansione m)	Crea una nuova mansione con i dati necessari e la passa alla funzione del Gestore DB per salvarla.

Guasto

Questa classe contiene tutte le informazioni che riguardano la segnalazione effettuata da un inquilino riguardo un guasto all'interno dell'appartamento in cui risiede.

Operazione	Descrizione
aggiornaStatoG (Guasto g)	Durante il processo di manutenzione, la funzione tiene aggiornato lo stato del guasto e valorizza i campi delle date di presa in carico e risoluzione.

Bolletta

La classe bolletta contiene i principali dati d'interesse per l'inquilino riguardo i costi delle utenze durante la sua permanenza in un appartamento.

Operazione	Descrizione
aggiornaStatoB (Bolletta b)	Una volta saldato l'importo, la bolletta viene contrassegnata come pagata.

Mansione

Questa classe fa riferimento alle mansioni svolte dai coinquilini, le quali possono essere occasionali o ricorrenti, come ad esempio le pulizie. Inoltre, l'attributo multiplo *oggetti* permette di registrare anche una lista della spesa.

Operazione	Descrizione
aggiornaStato ()	Una volta completata la mansione, viene contrassegnata come svolta.

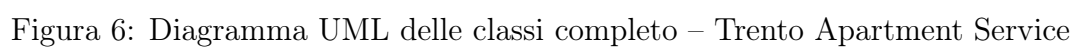
2.5 Diagramma completo

In questa sezione viene presentato l'intero diagramma UML delle classi con tutte le classi e relazioni non precedentemente esplicate.

Dipendente comunale

Questa classe si riferisce ai dipendenti del comune, i quali, in base a ruolo, dipartimento e permessi, hanno accesso a diversi dati statistici.

Operazione	Descrizione
visualizzaStats (...)	Richiede al Gestore DB la creazione di statistiche e stime su dati aggregati e anonimi riguardo gli appartamenti corrispondenti ai parametri di ricerca (opzionali).
esportaReport (...)	Genera un report con i dati statistici (come prezzo medio al mq, tasso di turnover, efficienza energetica ...) per gli appartamenti che rispettano gli eventuali filtri applicati.



3 Constraint OCL

In questa sezione vengono analizzati in linguaggio OCL i vincoli legati al diagramma delle classi. Questi vincoli rappresentano invarianti, precondizioni e postcondizioni sulle operazioni delle classi del sistema Trento Apartment Service.

3.1 Contratto - Validità delle date

La data di inizio del contratto deve essere precedente alla data di fine.

```
1 context Contratto
2 inv dateValide:
3     self.dataInizio < self.dataFine
```

3.2 Appartamento - Coerenza studentsOnly

Gli appartamenti per soli studenti devono avere almeno 2 stanze e 1 bagno.

```
1 context Appartamento
2 inv studentiCoerenza:
3     self.studentsOnly = true implies
4     self.numStanze >= 2 and self.numBagni >= 1
```

3.3 Recensione - Range valutazione

La valutazione deve essere compresa tra 1 e 5 stelle.

```
1 context Recensione
2 inv valutazioneValida:
3     self.valutazione >= 1 and self.valutazione <= 5
```

3.4 Amministratore - Univocità pIVA

Ogni amministratore deve avere una partita IVA univoca.

```
1 context Amministratore
2 inv pIvaUnica:
3     Amministratore.allInstances()->forAll(a1, a2 |
4         a1 <> a2 implies a1.pIVA <> a2.pIVA)
```

3.5 Guasto - Priorità e stato

Un guasto con priorità urgente non può essere in stato archiviato.

```
1 context Guasto
2 inv prioritàUrgente:
3     self.priorità = ReportPriority::URGENTE implies
4     self.stato <> ReportState::ARCHIVIATO
```

3.6 Bolletta - Periodo valido

Il periodo deve contenere esattamente 2 date: inizio e fine, con inizio < fine.

```
1 context Bolletta
2 inv periodoValido:
3     self.periodo->size() = 2 and
4     self.periodo->at(1) < self.periodo->at(2)
```

3.7 Camera - Prezzo positivo

Il prezzo mensile di una camera deve essere maggiore di zero.

```
1 context Camera
2 inv prezzoValido:
3     self.prezzo > 0
```

3.8 Login - Utente esistente

Per effettuare il login, l'email deve corrispondere a un utente registrato.

```
1 context Autenticatore::login(email: String, password: String)
2 pre utenteRegistrato:
3     UtenteRegistrato.allInstances()->exists(u |
4         u.email = email)
```

3.9 Creazione Contratto - Disponibilità appartamento

Un inquilino può firmare un nuovo contratto solo se l'appartamento ha stanze disponibili.

```
1 context Inquilino::aggiungiMansione()
2 pre appartamentoDisponibile:
3     self.contratto->select(c |
4         c.stato = ContrStatus::ATTIVO)->size() <
5     self.contratto.appartamento.numStanze
```

3.10 Registrazione - Creazione utente

Dopo la registrazione, deve esistere un nuovo utente con i dati forniti.

```
1 context Autenticatore::registrazione(
2     nome: String, cognome: String,
3     email: String, password: String)
4 post utenteCreato:
5     UtenteRegistrato.allInstances()->exists(u |
6         u.email = email and
7         u.nome = nome and
8         u.cognome = cognome)
```